have chosen a hash table or, if we could trade saving some memory for a certain loss of precision, we could even resort to a Bloom filter; since the requirement is maintaining an open, dynamic set, however, the one data structure providing the best compromise for all the operations would be a tree.

A simple binary tree could of course be enough to support all the operations provided by dictionaries; figure 6.1 shows a possible representation for an example of how these trees could look like, where we chose to show only a small part of the subtree containing a few similar words (we'll see why this is relevant in a few lines).

How fast can operations on such a tree be? Assuming the tree is balanced, its height will be logarithmic in the number of words it contains, and so for each call to `contains`, `insert`, `remove` etc... we'll need to traverse on average (and at most) `O(log(n))` nodes.